

Java HashMap

The Ultimate Key-Value Data Structure

CODEHIVE

CODEHIVE

1. What is a HashMap?

A HashMap is a data structure that implements the Map interface. It works on the principle of **Hashing**, allowing you to store data as **Key-Value pairs**.

Unlike a List, you don't access elements using an index (0, 1, 2). Instead, you use a unique Key (like a Country name) to find its corresponding Value (like a Capital city).

Key Characteristics:

- **Keys are Unique:** You cannot have two identical keys.
- **Values can be Duplicated:** "Germany" and "Austria" can both have "Vienna" (theoretically).
- **Unordered:** Does not maintain the order of insertion.
- **Nulls:** Allows one null key and multiple null values.

2. Basic Operations Example

This code demonstrates the primary ways to interact with a HashMap, from adding data to checking existence.

```
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {
        HashMap<String, String> capitalCities = new HashMap<>();

        // 1. Add Data (Key: Country, Value: City)
        capitalCities.put("England", "London");
        capitalCities.put("Germany", "Berlin");
        capitalCities.put("Norway", "Oslo");

        // 2. Access a value
        System.out.println(capitalCities.get("Germany")); // Berlin

        // 3. Conditional Check
        if(capitalCities.containsKey("Norway")) {
            System.out.println("Norway found!");
        }

        // 4. Removal
        capitalCities.remove("Norway");
    }
}
```

3. The "Big Four" Methods

The core of HashMap manipulation revolves around these four operations:

PUT `put(K key, V value)`: Inserts the pair. If the key already exists, it **overwrites** the old value.

GET `get(K key)`: Retrieves the value. Returns `null` if the key isn't there.

HAS `containsKey(K key)`: Returns a boolean. Highly useful for defensive programming.

SIZE `size()`: Returns the number of entries (key-value pairs) in the map.

4. Traversing the Map

Since there are no indexes, we iterate using the `keySet()` or the `entrySet()`.

The KeySet Approach

Useful when you only need keys or want to pull values manually.

```
for (String country : capitalCities.keySet()) {  
    System.out.println(country + " -> " + capitalCities.get(country));  
}
```

The EntrySet Approach (Efficient)

Best for large maps because it doesn't perform a `get()` lookup for every iteration.

```
for (Map.Entry<String, String> entry : capitalCities.entrySet()) {  
    System.out.println(entry.getKey() + ": " + entry.getValue());  
}
```

5. How it Works: Hashing & Buckets

Inside a HashMap is an array of "buckets." When you put ("England" , "London"):

1. Java calls "England" .hashCode () to get a number.
2. It uses this number to calculate an **index** in the internal array.
3. It stores the Entry object in that specific bucket.

When you call get ("England") , it repeats the math, jumps straight to the index, and retrieves the value instantly. This is why it's so fast!

6. Handling Collisions

What if two different keys (e.g., "KeyA" and "KeyB") produce the same bucket index? This is a **Collision**.

The Solution: "Separate Chaining"

Initially, Java stores colliding items in a **LinkedList** at that bucket index. As the list grows longer (exceeding 8 items), Java automatically converts that list into a **Red-Black Tree** to keep performance high.

7. Performance: $O(1)$ Search

Because the HashMap is designed to jump directly to an index using a hash, it is incredibly efficient.

Operation	Time Complexity
Search (get)	$O(1)$ Average
Insert (put)	$O(1)$ Average
Delete (remove)	$O(1)$ Average

*Worst case (many collisions) is $O(\log n)$ since Java 8.

8. Comparing Map Types

Just like Sets, there are different "flavors" of Maps depending on your needs.

Type	Order	Speed
HashMap	Unordered	Fastest
LinkedHashMap	Insertion Order	Fast
TreeMap	Sorted (Keys)	Moderate ($O(\log n)$)

9. Summary & Final Verdict

1. Use **HashMap** for general caching or lookup tables where order doesn't matter.
2. Always use meaningful objects as Keys (Strings and Integers are perfect).
3. Don't forget that Keys are case-sensitive ("USA" != "usa").
4. Declare using the interface: `Map<K, V> map = new HashMap<>();`

© 2026 CodeHive Academy

Java Collections Documentation - Issue #08